

Building and Auditing a Physical S7comm Intrusion-Detection Dataset

Thai Thi Minh Tu, Vuong Linh Thao, and Dang Le Dinh Trang

Le Quy Don Technical University, Hanoi, Vietnam

thaithiminhtu.338@gmail.com, thaolinh252512@gmail.com, trangdld@lqdtu.edu.vn

Abstract

Public intrusion-detection datasets for Siemens S7comm rarely combine traffic collected from physical hardware with process observations and construction metadata that let downstream users assess labeling and preprocessing quality. We present a physical S7comm dataset collected from a Siemens S7-1500 testbed over six sessions. Three independently recorded sources—network captures, PLC process logs, and attack-event records—are merged by nearest recorded timestamp and converted into 55,902 non-overlapping 2-second windows, including 47,460 benign and 8,442 attack windows from nine attack scenarios. The release defines windows, attack episodes, and collection sessions explicitly and provides episode-level grouping metadata for leakage-aware evaluation. A central contribution is the data-quality audit built into the release process. This audit identified and corrected three defects induced by collection and preprocessing: process-availability masking, cross-session imputation, and a retained collection artifact. We document their causes, corrections, and measured effects to provide a reusable procedure for other dataset builders. The audit also determines which released view suits which use: the network view is recommended for IDS benchmarking, while the process-only and fusion views are released primarily to support data-quality and modality-coverage research rather than a controlled modality comparison. The release includes extracted feature views, split definitions, preprocessing and reference-model code, and a data card describing the schema, labeling policy, timestamp-alignment policy, and known limitations.

Index Terms— industrial control systems, S7comm, intrusion detection dataset, PLC testbed, dataset construction, data quality

I. Introduction

Industrial Control Systems built around Siemens S7-family PLCs remain widely deployed in manufacturing, utilities, and distributed infrastructure. The S7comm protocol governing HMI-PLC and engineering-station communication was not designed with authentication or encryption by default. A 2026 taxonomic review of 23 public ICS/IIoT intrusion-detection datasets finds protocol coverage heavily skewed toward Modbus—appearing in 13 of 23 datasets against S7comm in only five—and that most datasets model short, isolated attack episodes rather than a reconnaissance-through-impact progression [1].

Three gaps follow. First, S7comm is under-served relative to its deployment footprint. Second, datasets that do target it are

typically collected on a single control process, making it impossible to tell whether a detector has learned protocol behavior or one program’s idiosyncrasies. Third, and most consequentially for whether a released dataset can be trusted, few public releases document how the data was built well enough for a downstream user to know what the labels, timestamps, and process fields mean, or when an evaluation protocol has silently leaked information across a split.

This paper describes the construction, organization, and quality audit of a physical S7comm intrusion-detection dataset. We focus on three aspects required for trustworthy reuse: coverage of benign and adversarial operating conditions, reproducible alignment and labeling of independently recorded data streams, and explicit documentation of collection- and preprocessing-induced artifacts. Building a dataset that couples network behavior with the resulting process state raises concrete engineering problems—keeping three independently running loggers on a common timeline, defining a window and grouping unit that does not leak an attack episode across a cross-validation split, and detecting when the collection pipeline itself has introduced an artifact a model could exploit instead of learning attack behavior. We treat solving and documenting these problems as the paper’s main contribution; baseline classifier results are reported afterward to validate the released features technically, and they do not constitute a benchmark claim.

Existing S7comm work has focused on protocol modeling [2], a fixed-process physical testbed [3], or attacks injected into a single-process capture [4] (Section II compares these in detail). What remains rare is a dataset that preserves the relationship between the sender, the command, the targeted memory area, and the resulting process-state change—since a single correctly addressed S7comm write can corrupt a control cycle without any protocol-level anomaly—while exercising more than one control application and being explicit about where collection falls short of that goal.

The main contributions of this work are:

1. A physical Siemens S7-1500 intrusion-detection dataset containing 55,902 non-overlapping 2-second windows collected across six sessions and nine attack scenarios.
2. A timestamp-based pipeline that combines network captures, PLC process observations, and attack-event records, together with explicit window, episode, and session definitions for leakage-aware evaluation.
3. A documented data-quality audit identifying three collection and preprocessing defects, tracing their operational

causes, and quantifying their effects before and after correction.

4. A reproducible release comprising extracted feature views, split definitions, preprocessing code, reference models, and a data card describing labels, fields, and known limitations.

We keep the multi-application claim narrow (Section VI). We scope this work to dataset construction, auditing, and release; detector benchmarking and controlled model comparison remain outside the study design.

II. Related ICS Datasets

S7comm protocol modeling. Kleinmann and Wool model periodic S7 traffic as a per-channel Deterministic Finite Automaton, flagging out-of-sequence messages and unexpected memory references [2]—structural regularity exploitable for anomaly detection, but without coupling to process-state change or a released, scenario-organized dataset.

Physical S7comm testbeds. Rodofile et al. release QUT S7comm from a physical mining/refinery testbed (nine hours, 13 attack types) [3]—the closest prior work in spirit, but single-process and, to our knowledge, without a documented leakage audit. Al-Duwairi et al. release S7-1500 incinerator captures with attacks injected post hoc rather than executed live [4]. Kellerer et al. release a physical-testbed S7comm dataset spanning two process contexts (battery storage, PV) with live protocol-level write attacks [10, 11].

Multi-protocol and multimodal ICS datasets. Gómez et al. build Electra from an electric traction substation using Modbus TCP and S7comm [5]. Dehlaghi-Ghadim et al. present an anomaly-detection dataset on a heterogeneous-protocol testbed [6]. Shafir et al. present ICS-MMOC3, pairing operational faults with cyber-attacks on a power-generation ICS and separating network-centric from process-centric modalities [12]. Zhou et al. present ICS-NAD, spanning three vendor platforms and two processes with 20 attack types [13]; vendor diversity there is a separate axis from process/application diversity (Table 1). SWaT [7] and WADI [8] remain the most widely used physical ICS benchmarks outside the S7/Modbus family; Stuxnet [9] remains the standard motivating reference for treating ICS network security as distinct from IT security.

We evaluate the datasets above against three criteria: physical S7 capture, network/process coupling, and leakage-aware construction documentation (Table 1). We did not identify an equivalent, fully documented leakage audit among them; that audit, alongside physical-S7comm/process-state coupling, is our distinguishing contribution. According to the taxonomy in [1], our dataset is a deliberately-injected physical-testbed release (their Realism tier 1, shared with SWaT, Rodofile, MSU-GP), with attack coverage spanning reconnaissance, unauthorized writes, setpoint/sensor tampering, a stealthy low-footprint write, and volume/fuzzing attacks (their Diverse tier, ≥ 5 categories); its 5.6:1 benign-to-attack ratio falls in their Moderate imbalance band with SWaT and ICS-Flow, not the Severe band of WADI or WUSTL-IIoT.

III. Dataset Construction

A. Physical Testbed and Threat Model

The testbed deploys a physical Siemens S7-1500 PLC (.211) executing real control logic, connected to a genuine engineering/HMI station (TIA Portal / WinCC Runtime V18, .31) over an isolated, segmented network. An attacker host (.32) runs Python, Scapy, and python-snap7. All hosts connect through an Aruba switch (.254) configured for ERSPAN GRE mirroring to a dedicated capture/logging host, which independently records network capture (PCAP), the PLC tag/process log, and the attack-event timeline used as audit ground truth; the three streams are merged by timestamp (Fig. 1). A physical PLC, rather than PLCSIM, improves the realism of cyclic scan timing and protocol-level responses under normal and adversarial conditions. Two control applications are instrumented and released—an urban traffic-light controller and a conveyor-belt sequential-logic process; a third (irrigation/water-pump) is implemented on the same PLC but not yet captured. The current release does not support an auditable per-application window breakdown because application identifiers were not preserved at window level (Section VI).

We assume an attacker with existing network access to the Field OT segment reaching the PLC’s S7 service on TCP/102; we do not simulate initial compromise. Out of scope: physical sabotage, bypassing a Safety PLC, firmware persistence, functional-safety evaluation. All experiments run on authorized equipment with dedicated test memory areas and a restore step after each write episode.

Inspection of the recorded traffic confirms that the HMI channel (TIA Portal/WinCC Runtime V18) uses S7CommPlus, while attacker traffic from python-snap7—a single host (.32), distinct from the HMI host (.31)—uses classic S7comm; the two are therefore distinguishable at the framing level independent of intent. Raw host identifiers and any undecodable S7CommPlus payload content are excluded from the feature matrix; however, L0/L1 retain volume, protocol-presence, framing, and directional/connection-pattern statistics. A network-only classifier may therefore partly exploit protocol variant or communication role instead of attack semantics. We treat this as an unresolved validity confound rather than assume it is benign, and the reported scores in Section V should be read accordingly; see Section VI.

B. Collection Schedule and Attack Scenarios

Table 2 catalogs the nine attack scenarios underlying the eight released classes, together with the measured window counts by phase. Day 1 is a benign baseline; Days 2–5 each exercise a subset of scenarios (reconnaissance-to-impact progression: enumeration/scan, then direct register writes, then setpoint/spoof/stealthy logic tampering, then volume/fuzzing attacks), targeting confirmed PLC memory offsets documented in the data card. Day 6 contains all nine scenarios at reduced rates and serves as a session-disjoint diagnostic under within-testbed schedule variation; we treat it as exploratory rather than a confirmatory domain-shift holdout (Section III-E).

Each scenario runs three episodes across Days 2–5 (randomized duration and inter-episode gap) plus one additional episode on Day 6 at 20–50% lower rate, for 36 attack episodes in total.

Table 1: Dataset comparison

Dataset	Phys.	S7	PCAP	Proc. vars.	Sess.-disj.
QUT S7comm [3]	Y	Y	Y	Y	N
Incinerator [4]	Y	Y	Y	N	N
Electra [5]	Y	Y+M	Y	N	N
Dehlaghi et al. [6]	Y	N	Y	Y	N.r.
Kellerer et al. [10]	Y	Y	Y	Y	N.r.
ICS-MMOC3 [12]	Y	Y	Y	Y	N.r.
ICS-NAD [13]	Y, 3 vend.	Part.	Y	N	N.r.
This work	Y	Y	Y (post-acc.) [†]	Partial [*]	Y (D6, expl.)

N.r. = not reported in the source publication; M = Modbus TCP; Sess.-disj. = session-disjoint evaluation split; expl. = exploratory rather than confirmatory (Section III-E). [†]post-acc. = full PCAP/PCAPNG release scheduled after acceptance (Section IV). ^{*}Partial process coverage; see Section III-E for the measured extent.

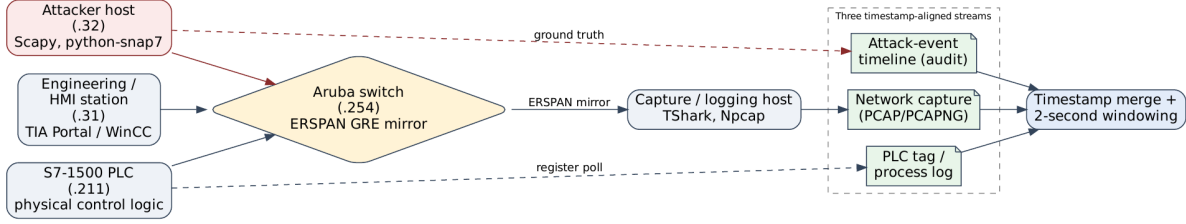


Figure 1: Testbed architecture. PLC, HMI, and attacker traffic are ERSPAN-mirrored to a capture host that independently records three streams, merged by timestamp into 2-second windows.

Live execution against the physical PLC, rather than post-hoc packet injection into an existing capture, is a further distinguishing feature of this release relative to [4].

Register-level effects were confirmed against PLC memory reads rather than assumed from the attack script. RWRITE toggles M5.0 from 0 to 1 to force a START transition (M5.1 from 0 to 1 forces STOP); SETP overwrites a specific setpoint Merker double-word offset (MD50/MD54/MD58/MD62) from its baseline value to an attacker-chosen value; SPF overwrites a sensor-mapped tag so the reported value diverges from the true sensor reading; STL’s STOP-bit write is the one effect independently confirmed in the released process log, moving the Q0 output register from 41 to 40 for the attack’s duration (Section III-E). ENUM, SCAN, FLD, and FUZZ are network-only by design and produce no expected process-state change. Exact memory offsets, confirmed before/after values, and per-scenario write rates are documented in the data card rather than reproduced here.

The six sessions span approximately 49.8 wall-clock hours from first to last recorded event; the released non-overlapping 2-second windows represent 31.1 of those hours. Windows were generated only while both packet capture and the labeled session timeline were active; the remaining 18.7 hours are overnight breaks and capture-stopped intervals, and no interval was excluded based on its label or feature values. Benign traffic covers startup, steady-state, shutdown, mode changes, sensor transitions, HMI polling, and legitimate engineering writes, retained deliberately as hard negatives.

C. Timestamp Merging, Windowing, and Labels

The three streams are merged by nearest recorded timestamp rather than a hardware-synchronized clock; we have not precisely measured the inter-host clock offset or the process-logger polling interval, so we describe the merge as timestamp-based rather than

Table 2: Collection and attack-scenario catalog

Scenario (label)	Tool / target effect	D1–5	D6	Total
Enumeration (ENUM)	py-snap7/Scapy; discover PLC resources	935	470	1,405
Scan (SCAN)	Scapy; probe S7 service reachability	438	462	900
Reg. write (RWRITE)	py-snap7; write M5.0 START/M5.1 STOP	866	365	1,231
Setpoint (SETP)	py-snap7; change process setpoint	829	253	1,082
Spoofing (SPF)	py-snap7; manipulate sensor value	943	299	1,242
Stealthy (STL)	py-snap7; STOP-bit write, Q0 41→40	442	397	839
Flood (FLD) ^a	Scapy; S7 exhaustion + SYN flood	894	317	1,211
Fuzzing (FUZZ)	Scapy; malformed TPKT/S7 payload	410	122	532
Benign	Startup, steady state, transitions, HMI polling, legitimate writes	42,857	4,603	47,460
Total		48,614	7,288	55,902

py-snap7 = python-snap7. ^aFLD aggregates two scenario IDs, S7 FLOOD (623 windows total) and SYN FLOOD (588 windows total), sharing one released label; the per-day split between the two sub-scenarios is in the data card. D1–5 totals include Day 1’s 7,348 benign-only windows.

claiming sub-second synchronization accuracy (Section VI).

A window is a fixed 2-second interval, stride 2 s, no overlap, labeled ATTACK if any attack interval overlaps it; rows within a configurable transition margin around attack boundaries (default 10 s) are additionally dropped, applied symmetrically regardless of label, to avoid ambiguous mixed-state windows. An episode is one warm-up→attack→cooldown→restore sequence, the grouping unit for cross-validation. A session is all data from one collection day. The effective grouping key combines the session and episode identifiers (session_id|episode_id). For attack windows, episode_id identifies one of the 36 attack episodes. Benign windows are chunked into 10-minute episode_id segments, giving 192 benign groups and 228 groups in total for the network and fusion views (49 for process-only, which contains only logger-covered rows). We call this episode-grouped cross-

validation rather than session-grouped: repeated episodes of a scenario share script and parameters, so grouping by episode prevents seeing the exact same episode in both folds, but not a different episode of the same scenario, script, and day.

D. Released Data Views and Split Metadata

Features are organized into four cumulative layers: L0—network volume/protocol statistics (115 features); L1—ICS protocol presence (+32); L2—S7 operation semantics (+45: read/write counts, CPU-control ops, addresses mapped to process-meaningful tags, e.g. M5.0→START); L3—process state (+132: `proc_*` register-polling features), for 324 features cumulative at L0+L1+L2+L3. Three feature views are released. The network view (L0–L2, all 55,902 windows) is the primary view for IDS experiments. The process-only view is restricted to the 10,771 rows with a real, logger-covered process observation (`proc_data_valid=1`). The fusion view retains every network window and fills `proc_*` values for windows without a real observation via a per-day backward-fill, so the imputation risk in Section III-E applies to fusion, not to process-only. Each view ships with window-level metadata (label, `session_id`, `episode_id`, and boundary flags) for grouped splitting and traceability, and with grouped-CV and Day-6 split definitions to reproduce the results in Section V.

Session/day identifiers, filenames, host identity, timestamps, and rule-derived flags are excluded from the feature matrix. Scenario and episode identifiers (`scenario_id`, `episode_id`) are used only for grouped-CV splitting and ground-truth labeling, never as model inputs; the attack-event log is used only for ground truth and audit. `proc_data_valid`, which marks whether the process logger was running for a window, is excluded from all three feature views (rather than only flagged) because, as Section III-E details, it is an indirect-leakage risk rather than a security-relevant signal.

E. Data-Quality Audit

Building a trustworthy multi-source dataset requires catching the pipeline’s own mistakes, not only the testbed’s. During internal testing, an early pipeline version produced $MCC = -0.154$ (Random Forest, process-only, Day 6)—a worse-than-chance, consistently-signed result that prompted an audit of labels and preprocessing rather than being accepted at face value. Table 3 summarizes the three defects the audit found and fully corrected before release. We frame these as quality-control findings that demonstrate the audit procedure’s effectiveness and are reusable by other dataset builders. They are not residual weaknesses of the released data: every released file already reflects the corrected pipeline.

The common enabling condition was operational: the process logger was started manually and covered only part of several attack periods. Two of the three defects arose from how missing observations were subsequently handled; the third arose from a preprocessing choice to retain the resulting logger-availability pattern as a model feature rather than excluding it. In practice, the logger started 13 minutes after the last Day-2 scan window, 36 minutes after the Day-3 write window, and only 10 seconds before the Day-5 attack period ended—so the register-write (RWRITE), flood (FLD), and setpoint (SETP) classes have

Table 3: Data-quality defects identified and corrected by the audit

Defect	Root cause	Effect	Correction
Incorrect availability mask	Mask computed after missing values filled	Imputed rows marked as observed	Compute mask before imputation
Cross-session propagation	Fill globally applied to ordered rows	Process values leaked across day boundaries	Restrict fill to within-day
Mask used as feature	<code>proc_data_valid</code> retained in feature matrix	Model could learn logger timing, not behavior	Exclude mask from all views

zero windows with a real process observation anywhere in their collection day.

The corrections changed Day-6 process-only MCC from -0.154 to $+0.374$ (Random Forest), 0.000 to $+0.376$ (Logistic Regression), and -0.007 to $+0.374$ (CatBoost). Nevertheless, only 1,698 of 8,442 attack windows (20.1%) contain a real process observation, and only the 839 stealthy (STL) windows (9.9% of all attack windows) carry a process signature that discriminates from benign at all: STL’s STOP-bit write changes the Q0 output register from 41 to 40, while every other observed class leaves Q0 at 41. We acknowledge that a `proc_data_valid=1`-only matched comparison would reduce the task to STL-only detection rather than a genuine multiclass validation. Addressing this coverage gap requires re-collecting data with the logger running for the full duration of every session—a different evaluation protocol on the current data cannot substitute—which we leave as future work. We report this residual coverage gap as part of the dataset’s documentation rather than a generic “future work” sentence: it is precisely the kind of constraint the audit is designed to surface, and it determines which released view—network, process, or fusion—suits a given downstream experiment.

IV. Data Records and Recommended Use

The release comprises: (i) three extracted feature views (network, process-only, fusion) as described in Section III-D, one row per window; (ii) split definitions (`episode_id/session_id` assignments) for episode-grouped CV and the Day-6 diagnostic split; (iii) preprocessing and training code (`merge_dataset.py`, `train_ml.py`); and (iv) a data card documenting the schema, window size and stride, label-precedence rule, timestamp-alignment policy, and the defects and corrections in Table 3. We recommend the network view as the primary view for binary or multiclass IDS experiments, episode-grouped CV on Days 1–5 for in-distribution model selection, and the Day-6 split as a diagnostic check on schedule shift (Section VI). The process-only and fusion views are recommended for data-quality and modality-coverage research, including reproducing or extending the audit in Section III-E; we do not offer them as a controlled fusion benchmark. Session/day identifiers, filenames, host identity, timestamps, rule-derived flags, and `proc_data_valid` are withheld from all three feature views for the leakage reasons given above; users needing these fields for other purposes should consult the data card. Extracted feature views, split definitions, code, and the data card are archived at Zenodo under a fixed DOI (<https://doi.org/10.5281/zenodo.21482630>); the

record is publicly visible but files are currently restricted pending release, with access available to reviewers on request and the full PCAP/PCAPNG captures to follow upon acceptance.

V. Technical Validation

We use four off-the-shelf classifiers (Random Forest, Logistic Regression, XGBoost, CatBoost) to verify that the released network features support reproducible intrusion-detection experiments. These models are not intended to establish a state-of-the-art benchmark. Because the process-only and fusion views differ in observation coverage and imputation (Section III-E), their results are reported separately, as diagnostics of the process-logging audit rather than a controlled comparison against the network view.

A. Network-Feature Validation

We apply a common evaluation protocol to the three released views: (1) episode-grouped cross-validation (StratifiedGroupKFold, 5 folds, grouping as in Section III-C) on Days 1–5; and (2) the Day-6 diagnostic split (randomized order, reduced rate), never used for threshold tuning or feature selection, but whose anomalous early score is what triggered the audit above. This subsection reports the primary network-view validation; Section V-B discusses the process-only and fusion results as audit diagnostics. Binary detection is primary; multiclass attribution is secondary. NaN/infinite values are replaced with zero; constant features and pairs with Pearson $|r| > 0.98$ are pruned per training fold only. The binary decision threshold is selected per outer split using grouped out-of-fold predictions maximizing F_β ($\beta = 2$, favoring recall); an in-sample threshold gave visibly different numbers and model rankings, so results throughout use the out-of-fold procedure (`fbeta_oof`). Results are means over five seeds (42–46), reported in Table 4.

Network-only in-distribution performance is fairly uniform (0.833–0.909 Macro-F1); the Day-6 gap is model-dependent (−0.239 to −0.255), and the ranking of which model degrades least depends on the threshold-calibration method as much as on the model itself. At the selected threshold, the recorded Day-6 operating points exhibit very high attack precision but low recall across all four models and both the network and fusion views, indicating conservative transfer from Days 1–5 rather than a well-calibrated operational alarm rate; full precision/recall tables per model and seed are released with the code. Day-6 FPR/hour stays below 3 windows/hour for every network and fusion model at this recall-favoring threshold; a full per-seed table is released with the code. A separate Day-5 temporal check (train Days 1–4, test Day 5; 11,091 windows, 88.2% benign, only FLD/FUZZ present) serves as an auxiliary sanity check confirming zero metadata leakage and zero group overlap.¹ Day 5 and Day 6 also differ in attack composition; this compositional difference, more than schedule shift alone, likely explains the difficulty gap. A layer-wise feature ablation (L0 through L0+L1+L2+L3, Days 1–4) is released with the code; process state neither uniformly helps nor hurts binary detection.

¹RF and XGBoost reach ≥ 0.998 Macro-F1 on Day 5, which contains only packet-rate-separable FLD/FUZZ traffic; this result provides little evidence of semantic transfer.

This release’s multiclass task is currently hard for one representative model (CatBoost, Day 6): three of eight classes (RWRITE, ENUM, FUZZ) have zero recall under argmax, macro-averaged F1 is 0.206 (network-only) versus 0.235 (fusion), and STL’s fusion F1 of 1.000 is a documented artifact of the Q0-register imputation (Section III-E) and should not be read as a working detector. The full per-class breakdown for all four models is released with the code.

B. Process-Logging Diagnostics

Fusion shows no single direction of effect relative to network-only: lower for CatBoost/Random Forest, higher for XGBoost, close for Logistic Regression (Table 4). This pattern is consistent with the incomplete process coverage in Section III-E; it does not indicate a general property of fusion. All four process-only models return the same Day-6 score (0.547–0.548): feature importances show `proc_q0_raw_hex_mean` dominates every model, so the decision rule effectively reduces to a threshold on that one register, a narrow signature rather than a demonstration of broader process-state modeling. Given the process-coverage limits established in Section III-E and the multiclass advantage traced entirely to the STL artifact, this experiment cannot answer whether process fusion helps in general; it establishes only that any fused score should be audited per model and per class before being believed.

VI. Limitations and Conclusion

Process coverage. As established in Section III-E, most attack windows have no real process observation, concentrated exactly in the classes (RWRITE, FLD, SETP) where process impact would be most informative; fusion relies on same-day backward-filling and does not support a general conclusion about process fusion. We acknowledge this requires re-collection with synchronized logging, which we leave as future work.

Host, protocol-variant, and timing confounds. The host/protocol-variant confound from Section III-A is one of three related gaps acknowledged here. We have not precisely measured the inter-host clock offset underlying the timestamp merge (Section III-C), and the release has no `application_id` field, so it does not support a per-application breakdown or a multi-application generalization claim. Bounding these risks requires a cross-attacker-host evaluation, a targeted directionality/protocol-variant ablation, and hardware-synchronized logging; we leave this as future work.

Day-6 evaluation status. Day 6 serves as a post-audit diagnostic session because its results informed revisions to the pipeline. A confirmatory evaluation would require a session sealed before any pipeline inspection (e.g. a Day 7, locked before the pipeline is touched again); the Day-5 temporal check is the closest available substitute. “Shift” means within-testbed schedule shift only: Days 1–6 share PLC, attacker host, topology, and tooling. Episode-grouped CV measures within-schedule generalization; independent deployment episodes remain untested.

We release a physical S7comm intrusion-detection dataset containing 55,902 timestamp-merged windows collected across six sessions. Its central contribution is a data-quality audit demonstrated in the release process itself: it identifies three

Table 4: Binary detection under grouped out-of-fold threshold calibration (fbeta_oof), mean $\pm$ std over five seeds. Fusion/process rows are audit diagnostics and should not be interpreted as a modality comparison (see text).^a

View	Model	Grp. CV Macro-F1	Day-6 Macro-F1	Δ F1	Grp. CV MCC / Day-6 MCC
network	CatBoost	0.909 $\pm$ 0.002	0.659 $\pm$ 0.006	-0.250	0.831 $\pm$ 0.003 / 0.477 $\pm$ 0.007
network	Log. Reg.	0.833 $\pm$ 0.011	0.595 $\pm$ 0.000	-0.239	0.702 $\pm$ 0.013 / 0.396 $\pm$ 0.000
network	XGBoost	0.903 $\pm$ 0.001	0.649 $\pm$ 0.006	-0.255	0.821 $\pm$ 0.002 / 0.463 $\pm$ 0.007
network	Rand. Forest	0.900 $\pm$ 0.003	0.662 $\pm$ 0.005	-0.239	0.817 $\pm$ 0.005 / 0.479 $\pm$ 0.005
fusion ^b	CatBoost	0.922 $\pm$ 0.003	0.626 $\pm$ 0.005	-0.297	0.854 $\pm$ 0.005 / 0.436 $\pm$ 0.007
fusion ^b	Log. Reg.	0.910 $\pm$ 0.002	0.621 $\pm$ 0.018	-0.289	0.830 $\pm$ 0.003 / 0.430 $\pm$ 0.022
fusion ^b	XGBoost	0.919 $\pm$ 0.003	0.673 $\pm$ 0.010	-0.246	0.847 $\pm$ 0.006 / 0.495 $\pm$ 0.012
fusion ^b	Rand. Forest	0.918 $\pm$ 0.003	0.634 $\pm$ 0.022	-0.284	0.847 $\pm$ 0.005 / 0.446 $\pm$ 0.028
process ^b	CatBoost	0.389 $\pm$ 0.027	0.547 $\pm$ 0.000	+0.158	0.116 $\pm$ 0.043 / 0.370 $\pm$ 0.003
process ^b	Log. Reg.	0.402 $\pm$ 0.000	0.548 $\pm$ 0.000	+0.146	0.141 $\pm$ 0.001 / 0.376 $\pm$ 0.000
process ^b	Rand. Forest	0.393 $\pm$ 0.018	0.547 $\pm$ 0.000	+0.155	0.121 $\pm$ 0.025 / 0.372 $\pm$ 0.001
process ^b	XGBoost	0.401 $\pm$ 0.001	0.548 $\pm$ 0.000	+0.146	0.135 $\pm$ 0.004 / 0.374 $\pm$ 0.000

^aValidation results, not a state-of-the-art benchmark (Section V). ^bFusion and process-only views are audit diagnostics (Section III-E): process-logger coverage is incomplete, and same-day backward-filling in the fusion view fabricates process values for windows with no real observation.

commonly-overlooked pipeline defects, corrects them, quantifies their effect, and defines which released view suits which downstream use. Network features support reproducible reference experiments, while the process-only and fusion views serve data-quality and modality-coverage research. The released data, split definitions, preprocessing code, and data card provide a transparent, reusable basis for future S7comm IDS work; the priority for that future work is re-collecting with the process logger running for the full session, paired with a frozen, previously unseen evaluation session.

References

- [1] A. Termanini, H. Bourdouce, D. Al-Abri, and A. Al Maashri, “Intrusion Detection Datasets for IIoT and ICS: A Taxonomic Review with a Decision-Aid Scoring Rubric,” *Sensors*, vol. 26, no. 13, Art. no. 4099, 2026.
- [2] A. Kleinmann and A. Wool, “Accurate Modeling of the Siemens S7 SCADA Protocol for Intrusion Detection and Digital Forensics,” *J. Digital Forensics, Security and Law*, vol. 9, no. 2, pp. 37–50, 2014.
- [3] N. R. Rodofile, T. Schmidt, S. T. Sherry, C. Djamaludin, K. Radke, and E. Foo, “Process Control Cyber-Attacks and Labelled Datasets on S7Comm Critical Infrastructure,” in *Information Security and Privacy (ACISP 2017)*, LNCS vol. 10343, Springer, 2017, pp. 452–459.
- [4] B. Al-Duwairi, A. Shatnawi, A. Al-Hammouri, and M. Ababneh, “Dataset of SCADA traffic captures from a medical waste incinerator with injected cyberattacks,” *Data in Brief*, vol. 63, Art. no. 112294, 2025.
- [5] A. L. Perales Gómez et al., “On the Generation of Anomaly Detection Datasets in Industrial Control Systems,” *IEEE Access*, vol. 7, pp. 177460–177473, 2019.
- [6] A. Dehlaghi-Ghadim, M. Helali Moghadam, A. Balador, and H. Hansson, “Anomaly Detection Dataset for Industrial Control Systems,” *IEEE Access*, vol. 11, pp. 107982–107996, 2023.
- [7] A. P. Mathur and N. O. Tippenhauer, “SWaT: A Water Treatment Testbed for Research and Training on ICS Security,” in *CySWater*, 2016, pp. 31–36.
- [8] C. M. Ahmed, V. R. Palleti, and A. P. Mathur, “WADI: A Water Distribution Testbed for Research in the Design of Secure Cyber Physical Systems,” in *CySWATER’17*, ACM, 2017, pp. 25–28.
- [9] T. M. Chen, “Stuxnet, the real start of cyber warfare? [Editor’s Note],” *IEEE Network*, vol. 24, no. 2, pp. 2–3, 2010.
- [10] N. Kellerer, G. Sanchez Collado, H. Alberto, V. Hagenmeyer, and G. Elbez, “S7 Data Modification Attacks using an Industrial Control System Testbed,” Zenodo, May 2025. doi: 10.5281/zenodo.15373938.
- [11] N. Kellerer, G. Sanchez, H. Alberto, V. Hagenmeyer, and G. Elbez, “Attacks on the Siemens S7 Protocol Using an Industrial Control System Testbed,” in *Proc. 16th ACM Int. Conf. Future and Sustainable Energy Systems (e-Energy ’25)*, 2025, pp. 770–779. doi: 10.1145/3679240.3734645.
- [12] L. Shafir et al., “ICS-MMOC3: Exploring Operational Faults and Cyber Attacks with a Realistic Multi-Modal Power Generation ICS Dataset,” in *Proc. 12th ACM Cyber-Physical System Security Workshop (CPSS 2026)*, 2026, pp. 63–76. doi: 10.1145/3775042.3807883.
- [13] X. Zhou et al., “A dataset collected in real-world industrial control systems for network attack detection,” *Scientific Data*, vol. 13, Art. no. 399, 2026. doi: 10.1038/s41597-026-06738-x.